

Avaliação Técnica

Radar Ofertas · Node.js / TypeScript · Julho 2026 · Análise pós Fases 1–4 (escalabilidade)



Resumo executivo

O **Radar Ofertas** é um bot de coleta e publicação de ofertas do Mercado Livre com painel administrativo, filas BullMQ, PostgreSQL/Prisma e envio multi-canal (WhatsApp/Telegram). A base em `src/` segue organização por domínio com separação clara entre regras de negócio, integrações e orquestração.

Após as Fases 1–4 de escalabilidade (pool de filas, multi-conta runtime, contas em Prisma, manager stateless), o projeto evoluiu de um MVP operacional para uma base sólida single-tenant com caminho claro para N contas. Os maiores gaps restantes são operacionais (spawn por conta, token Telegram) e de **segurança** (painel sem CSRF, auth opcional, XSS em atributos JS) — adequado para rede confiável/VPN, não para exposição pública sem hardening.

~102 arquivos src/

~62 arquivos manager/

12 testes unitários

TS strict

CI: tsc + test

1. Arquitetura — 8,0 / 10

Pontos fortes

- **Processos separados:** collector (`app.ts`), workers por canal (`worker.ts` , `worker-telegram.ts`), manager — cada um com entry point e fila própria.
- **Contrato de canal:** `ChannelPublisher` em `channels/types.ts` — sender genérico (`jobs/sender.ts`) sem `if (whatsapp)` .
- **Delivery-first:** `dispatchOffer` abre `offerDelivery` antes de enfileirar — rastreabilidade e idempotência.
- **Fan-out multi-conta:** enfileiramento por (`canal` , `accountId`) com filas nomeadas (`getSenderQueueName`).
- **Config runtime:** score, brand, horários, templates em tabela `settings` com cache e fallback ENV.
- **Scraping híbrido:** HTTP primeiro, Playwright fallback, circuit breaker — decisão documentada e implementada.
- **Manager como UI:** MVC server-rendered reutiliza `src/` ; regras de oferta não vivem no painel.

Pontos fracos

- Login ML (Playwright) permanece stateful no manager — não replicável em cluster.
- Telegram multi-conta incompleto: token ainda lê `env.TELEGRAM_BOT_TOKEN` global.
- Collector + auto-messages no mesmo processo — acoplamento aceitável, mas limita escala independente.
- `telegram-publisher.ts` legado coexistindo com `publisher-factory.ts` .

Critério	Avaliação
Separação de domínios	Excelente — offers/, mercado-livre/, accounts/, channels/
Modelo de processos	Muito bom — Docker Compose com serviços independentes
Abstrações	Bom — ChannelPublisher, ServiceDeps, repository pattern
Acoplamento residual	Moderado — manager/process-model, Telegram token global

2. Qualidade de código — 7,5 / 10

Pontos fortes

- **TypeScript strict:** `strict: true` , `noUnusedLocals` , `verbatimModuleSyntax` .
- **ENV com Zod:** validação centralizada em `config/env.ts` com lazy proxy e stub para testes.

- **Testabilidade:** `ServiceDeps` injetável; `service.test.ts` com 10+ casos.
- **Validação de contas:** Zod em `account-config.ts` — shape tipado na leitura/gravação.
- **Error handling:** `DelayedError` para pacing, circuit breaker ML, retries BullMQ (5x, backoff 30s).
- **Logger centralizado:** pino em `utils/logger.ts` + log-store Redis.
- Sem `@ts-ignore` ou `as any` espalhados — disciplina razoável.

Pontos fracos

- `tsc` falha em ~14 erros (Baileys types, cheerio, manager unused imports) — gate de CI configurado mas código não limpo.
- 12 testes unitários — bom para domínio crítico, mas sem cobertura de `jobs/`, `whatsapp/`, `manager/`.
- Sem ESLint/Prettier — estilo depende de convenção manual.
- `npm test` com glob falha no Windows (PowerShell).
- `accounts/repository.ts` silencia erro de DB e cai para ENV — pode mascarar falhas.

Área testada	Arquivos
Ofertas / score / sampling	<code>service.test.ts</code> , <code>sampling.test.ts</code> , <code>message-template.test.ts</code>
Mercado Livre	<code>parser</code> , <code>circuit-breaker</code> , <code>category-url</code> , <code>coupon-parser</code>
Contas / Redis	<code>account-config.test.ts</code> , <code>redis-state.test.ts</code>
Sem testes	<code>jobs/</code> , <code>whatsapp/</code> , <code>telegram/</code> , <code>manager/</code>

3. Organização — 8,5 / 10

Pontos fortes

- Estrutura por domínio previsível — alinhada a `.cursor/rules/architecture.mdc`.
- Naming consistente: `kebab-case` em arquivos, `camelCase` em funções, `PascalCase` em tipos.
- Imports NodeNext com extensão `.js` — padrão ESM correto.
- Manager MVC bem decomposto: controllers finos, models por página, views em seções/components.
- Documentação interna rica em `.cursor/docs/` — architecture, accounts, queues, manager, deployment.
- Migrations versionadas; schema Prisma com índices adequados.

Pontos fracos

- `manager/routes/` é thin re-export de `http/routes/` — camada duplicada.
- Build `tsc` compila só `src/`; manager roda via `tsx` — sem artefato compilado do painel.
- Alguns arquivos legados coexistem com factories novas (publishers).

Domínio	Arquivos (~)	Responsabilidade
mercado-livre/	16	Scraping, afiliado, cupons
offers/	12	Regras de negócio, persistência
accounts/	9	Multi-conta, paths, Zod
channels/	6	Contrato + publishers
manager/	62	Painel MVC (~54 rotas)

4. Escalabilidade — 7,0 / 10

O que escala bem (pós Fases 1–4)

- **Pool de filas BullMQ:** `getQueue()` reutilizável — sem overhead de criar/fechar Queue a cada enqueue.
- **Filas por conta:** `offer-sender-{accountId}` — isolamento de jobs por conta.
- **Contas em Prisma:** tabela `accounts` com validação Zod — base para FKs futuras.

- **Manager stateless:** `MANAGER_CAN_SPAWN_WORKERS=false` no Docker; status via Redis + `owner.lock`.
- **QR/status no Redis:** `radar:connect:wa:{accountId}` — painel replicável atrás de load balancer.
- **Heartbeat worker:** `radar:worker:{channel}:{accountId}` — detecção cross-process.
- **Delivery por conta:** unique `(offer_id, channel, account_id)` — fonte da verdade granular.

Gargalos e limites

- **WhatsApp:** 1 sessão Baileys = 1 processo — não escala horizontalmente no mesmo número.
- **Collector singleton:** um worker `offer-collector`; Playwright é CPU/RAM pesado.
- **Pacing in-process:** `lastPacedSendAt` em memória — race se `senderConcurrency > 1`.
- **Spawn por conta:** painel ainda não sobe workers com `WORKER_ACCOUNT_ID` — manual via Docker/terminal.
- **Caches locais:** accounts, queue-config, metrics — inconsistentes entre réplicas sem Redis.
- **Auth manager:** `MANAGER_TOKEN` opcional — acesso livre se ausente.

Componente	Escala horizontal?	Nota
Collector (app)	Parcial	Repeat job único; scraping pesado
Worker WhatsApp	Não (por sessão)	Lock de dono + Baileys
Worker Telegram	Sim (por conta)	Bot API stateless
Manager	Sim (pós Fase 4)	Redis + spawn desligado
PostgreSQL	Sim	Prisma + índices adequados
Redis	Sim	BullMQ + estado compartilhado

5. Segurança — 5,5 / 10

O projeto foi desenhado como ferramenta operacional para uso local ou em rede confiável (VPN). Para exposição na internet pública, há débitos significativos que precisam ser endereçados antes de produção.

Pontos fortes

- **ORM parametrizado:** Prisma em todo acesso a dados — sem SQL injection em regras de negócio. Raw queries só em `preflight.ts` com tagged templates.

- **Validação de entrada:** Zod em `config/env.ts` e `account-config.ts` — shape tipado na leitura/gravação.
- **Escape HTML server-side:** `escapeHtml()` em views do manager — mitiga XSS em conteúdo renderizado.
- **Path traversal bloqueado:** `manager/http/static.ts` normaliza path e valida prefixo do diretório público.
- **Lock de sessão WhatsApp:** `owner.lock` impede dois processos na mesma sessão Baileys — reduz hijack acidental.
- **Segredos fora do código:** credenciais em `.env` (não commitado); `.env.example` sem valores reais.
- **Auth opcional documentada:** `MANAGER_TOKEN` com Bearer ou query param — consciente do trade-off dev vs prod.

Vulnerabilidades e riscos

Área	Risco	Evidência
Autenticação	Alto se exposto	<code>isAuthorized()</code> retorna <code>true</code> quando <code>MANAGER_TOKEN</code> não está definido — painel aberto por padrão
CSRF	Alto	Nenhum token CSRF nos ~20 POSTs destrutivos (delete oferta, contas, workers, settings)
XSS	Médio	<code>escapeHtml</code> não escapa <code>'</code> — <code>accounts.ts</code> L60: <code>onsubmit="confirm('...\${label}')"</code> quebrável com aspas no nome da conta
Token em URL	Médio	<code>?token=</code> vaza em logs de servidor, histórico do browser e header Referer
DoS	Médio	<code>readFormBody()</code> sem limite de tamanho — POST grande pode esgotar memória
Segredos em disco	Médio	Credenciais WhatsApp (<code>data/auth_*</code>), sessão ML e tokens Telegram em <code>accounts.config</code> JSON — sem criptografia at rest
Redis/Postgres	Médio em dev	Docker Compose expõe portas 5432/6379 no host; Redis sem senha por padrão; QR WhatsApp legível em <code>radar:connect:wa:*</code>
Headers HTTP	Baixo–Médio	Sem CSP, HSTS, X-Frame-Options ou X-Content-Type-Options no manager
Rate limiting	Baixo–Médio	Nenhum throttling em rotas do painel ou APIs JSON (<code>/manager/api/*</code>)

Client-side JS	Baixo	<code>innerHTML</code> em <code>settings.js</code> e <code>polling.js</code> — dados majoritariamente controlados, mas padrão arriscado
----------------	--------------	---

Superfície de ataque por componente

Componente	Exposto?	Proteção atual
Manager HTTP	Sim (porta 3000)	Token opcional; sem CSRF/rate limit
PostgreSQL	Dev: sim (5432)	Senha via ENV; Prisma ORM
Redis	Dev: sim (6379)	Sem auth default; estado QR/worker legível
Workers	Não (sem HTTP)	Processo interno; sessões em disco
Collector	Não (sem HTTP)	Só consome filas
Scraping ML	Outbound	User-agent configurável; circuit breaker

Recomendações prioritárias (segurança)

1. `MANAGER_TOKEN` **obrigatório em produção** — validar no preflight profile `manager` ; rejeitar boot sem token se `NODE_ENV=production` .
2. **CSRF tokens** em todos os POSTs destrutivos — especialmente delete de ofertas e contas.
3. **Corrigir XSS em** `accounts.ts` — escapar `'` em `escapeHtml` ou remover `onsubmit` inline (usar `data-*` + JS).
4. **Migrar auth para cookie HttpOnly** — eliminar `?token=` na URL; propagar sessão nos forms.
5. **Limitar** `readFormBody` — cap de 1 MB; retornar 413 se exceder.
6. **Redis/Postgres** — não expor portas em produção; Redis com `requirepass` ; rede interna Docker.
7. **Headers de segurança** — CSP, X-Frame-Options, HSTS via reverse proxy (nginx/Caddy) ou middleware.

Comparativo — antes vs depois das Fases 1–4

Aspecto	Antes	Depois
Filas BullMQ	Criar/fechar Queue a cada enqueue	Pool reutilizável (<code>getQueue</code>)
Multi-conta	Só dispatch; worker usava default	Fila + sender + publisher por accountId
Contas	JSON em settings	Tabela Prisma + Zod
Manager	Estado em memória (workers, QR)	Redis + owner.lock; spawn condicional
Escalabilidade manager	1 réplica	N réplicas possíveis

Top 6 prioridades

1. **Endurecer segurança do painel** — CSRF, `MANAGER_TOKEN` obrigatório, fix XSS em `accounts.ts`.
2. **Corrigir** `tsc` — erros Baileys/cheerio/manager bloqueiam gate de qualidade.
3. **Spawn workers por conta** — `process-model.ts` + templates Docker com `WORKER_ACCOUNT_ID`.
4. **Telegram multi-conta** — `botToken` por conta no publisher.
5. **Expandir testes** — jobs/sender, integração queue; glob cross-platform.
6. **Limitar** `readFormBody` e adicionar rate limiting no manager.

Conclusão

O Radar Ofertas apresenta **arquitetura madura para o escopo atual** — coleta ML, filas, envio multi-canal e painel administrativo — com decisões conscientes no código (delivery antes do enqueue, filas por canal/conta, affiliate link lazy, scraping híbrido). A organização é um dos pontos mais fortes do projeto.

A qualidade de código é boa no núcleo (`src/`), com TypeScript rigoroso e testes nos domínios críticos, mas perde pontos por dívidas de tipagem no manager/Baileys e cobertura estreita fora de `offers/` e `mercado-livre/`.

A escalabilidade melhorou significativamente com as Fases 1–4, mas limites reais permanecem: WhatsApp é intrinsecamente single-process por sessão, o collector é singleton, e multi-conta ainda exige operação manual para contas adicionais.

Em **segurança**, o projeto está adequado para uso em ambiente controlado (localhost, VPN, rede interna), mas **não está pronto para exposição pública** sem implementar CSRF, auth obrigatória, correção de XSS e isolamento de Redis/Postgres. A nota 5,5 reflete essa distinção: boas práticas no núcleo (Prisma, Zod, escape HTML), mas superfície de ataque significativa no painel administrativo.